

CITS3002 Computer Networks Project Report

Group Members

Student Name	Student Number
Kathleen Isabella	24091081
Kelly Valencia	24540356

Introduction

The purpose of this group project is to simulate a simplified network that demonstrates how Layer 2 (Data Link Layer), Layer 3 (Network Layer), and Layer 4 (Transport Layer) work together to deliver data from one host to another through a router, where each layer encapsulates the data as it moves down the stack.

Code Structure and File Organization

`config.py`

This file defines the fixed parameters for the simulation. This includes IP addresses, MAC addresses, Routing Tables, ARP (Address Resolution Protocol) Tables, and other constants from the layers.

The other constants from each layer were taken from the Protocol Header Definitions.

The IP and MAC addresses were taken from ‘IP Addressing Scheme’ and ‘MAC Addressing Scheme’ respectively.

The Routing Tables were created from the Routing/Forwarding functionality of the Network Layer, where “Each node must maintain a routing table that maps the destination IP address to the outgoing interface and next-hop IP addresses.” The outgoing interface name used was `eth0` and `eth1` since `eth0` is a common (Linux) designation for the first (Ethernet) network interface on a system and it would make more sense for Host B to view its network interface as `eth0` rather than `interface2` since Host B doesn’t know how many interfaces Router R1 has. An additional data structure, `R1_INTERFACE_TO_MAC` and `R1_MAC_TO_IFACE` was created to avoid hardcoding in the MAC address when routers receive frames as well as referring to them via `eth0` and `eth1`.

The ARP tables, although not explicitly mentioned, were created from the MAC Addressing functionality of the Data Link Layer, where “Each node must maintain a table that maps the next hop IP address (provided by the network layer) to the MAC addresses.”

`protocol.py`

This file defines the classes for each layer as well as their Protocol Header Definitions:

- `Frame` for Layer 2
- `Packet` for Layer 3
- `Segment` for Layer 4

This is also where the Encapsulation functionalities of each layer occurs as well as the Checksum Computation and Verification for the Segments.

devices.py

This file implements the **Host** and **Router** classes.

Host handles segmentation, route lookup, ARP lookup, encapsulation, sending, receiving, and rdt2.2 ACK logic.

Router handled MAC-based interface selection, MAC learning, TTL decrement, routing lookup, re-encapsulation, and forwarding.

There is also a helper function `ip_in_network()` that checks whether an IP address belongs to the network (specifically using /24 prefix).

main.py

This file is the main entry point for execution as it builds Host A, Host B, and Router R1. It registers the devices in `NETWORK_DEVICES` and creates a data payload to start the sending sequence from Host A to Host B. The payload is determined by the command-line argument specifying the data size, with input validation and error handling being implemented.

Code Design and Purpose of the Main Components

Host

`send_data()` breaks the application data into 500-byte chunks, as specified in the Execution Requirements for UDP-like segments. It then encapsulates each chunk of data into a **Segment** to send to the Transport Layer via Stop-and-Wait protocol.

Each **Segment** also includes the source port and destination port to simulate port-based communication. Although Application Layer is not part of this simulation, the ports are still added to demonstrate Transport Layer functionality.

The checksum of the **Segment** is computed inside the segment itself to support error detection.

This should fulfill the Transport Layer functionality for:

- Data segmentation
- Transport segment construction and encapsulation
- Port-based communication
- Checksum computation

`send_segment()` receives the **Segment** and constructs and encapsulates it into a **Packet** which specifies the source and destination IP addresses.

It then uses the routing table to determine the next-hop IP address and forwards the **Packet** to the Data Link Layer for transmission.

This should fulfill the Network Layer functionality for:

- Packet creation and encapsulation
- IP Addressing
- Routing/Forwarding decisions

`send_packet()` receives the **Packet** and looks up in the ARP table for the next-hop MAC address using the given next-hop IP address.

Then it creates and encapsulates the **Packet** into a **Frame** and specifies the source MAC address and destination MAC address (the next-hop MAC address).

It then forwards the **Frame** to the destination MAC address.

This should fulfill the Data Link Layer functionality for:

- Frame creation and encapsulation
- MAC addressing
- Switching/Forwarding

`receive_frame()` receives the **Frame** from the Data Link Layer and learns the incoming source MAC address. It then takes the **Packet** payload from the **Frame** and delivers it to the Network Layer. This should fulfill the Data Link Layer functionality for frame delivery of data to the upper layer.

The Network Layer which will then check for the destination IP address. If the **Packet** is meant for the end-host receiver, then it will take the **Segment** from the **Packet** payload and deliver it to the Transport Layer. This should fulfill the Network Layer functionality for packet delivery of data to the upper layer.

`receive_segment()` receives the **Segment** from the Network Layer and proceeds to do rdt2.2 logic. It first verifies the checksum to see if it needs to be discarded or not. Then it checks for valid DATA segments to see if it needs to resend the last ACK due to corruption or duplication. If it is valid, then it delivers the valid data to the Application Layer and sends an ACK Segment with the same sequence number to the Network Layer.

This should fulfill the Transport Layer Functionality for:

- Data delivery
- Reliable Data Transfer 2.2
- Checksum/Error Detection

Router

`receive_frame()` receives the **Frame** from Data Link Layer and determines the incoming interface from the destination MAC address as well as learning the source MAC address.

Then it extracts the **Packet** from the **Frame** and delivers it to the Network Layer, which will then read the destination IP address.

The **Packet** TTL will be decremented for every router hop it makes as well as dropping expired packets.

The routes are being decided based on the Routing Table using the destination IP address as it determines the next-hop IP address and the outgoing interface. Then the router will forward the **Packet** to the Data Link Layer.

The destination MAC address will be looked up from the next-hop IP address in the ARP table. Then the **Packet** will be re-encapsulated into a new **Frame** with the new destination MAC address and forwarded to the correct host.

Protocol Classes

Each class has a `total_size()` method that fulfills the functionality of length handling. Since the lengths are derived from header and payload size, it is better to get the size using a method so the size is always computed from the actual structure and values stored in `self.length` or `self.total_length` would not be inconsistent.

Message type

In `main.py`, the message data is transmitted as raw bytes (`data = b"X" * message_size`) as network layers typically store payloads in bytes. This means that the data does not need anymore encoding and can also be easily used in byte-level operations such as segmentation or checksum computation.